

第五章 软件包、仓库与基础系统维护

软件题不止要求“安装成功”，还要确认仓库来源、包版本、文件落点和服务入口。RPM 数据库描述已安装状态，DNF 使用仓库元数据求解依赖并执行事务；repo 文件正确也不等于远端内容可达。本章从仓库可用性开始，沿事务、包文件和服务 unit 完成验收。

[概念] RPM package 是带元数据、文件和脚本的安装单元，RPM 数据库记录本机已安装包；repository 提供包和 repodata；DNF 读取多个仓库、解析依赖、选择版本并调用 RPM 完成事务。

[概念] 包名、NEVRA (Name-Epoch-Version-Release-Architecture)、文件路径和命令名是不同查询入口。命令缺失时先从路径反查提供包；文件已存在时可从 RPM 数据库反查所属包。

[操作语义] `dnf install/remove/upgrade` 改变包事务；`dnf search/info/list/repoquery/provides` 查询仓库或包元数据；`rpm -q` 查询已安装数据库，`rpm -V` 比较已安装文件与包元数据。

[操作语义] `dnf repolist` 观察启用仓库，`dnf makecache` 实际获取元数据；repo 文件中的 `baseurl`、`enabled`、`gpgcheck` 和 `gpgkey` 分别描述位置、启用、签名校验和密钥入口。

[知识专题] RPM 已安装状态与 DNF 仓库状态

① [知识点] 查询对象决定命令入口

`rpm -q <PACKAGE>` 查询本机是否安装，`rpm -qi` 看已安装信息，`rpm -ql` 列包文件，`rpm -qf <PATH>` 从现有文件反查所属包。`dnf info` 默认可能同时显示 installed 和 available 信息；`dnf list --installed` 或 `--available` 明确范围。

```
rpm -q httpd                # 已安装包状态
rpm -ql httpd | grep -E 'systemd|conf' # 包内文件
rpm -qf /usr/sbin/httpd      # 文件所属包
dnf repoquery -l httpd       # 仓库包文件列表
```

`repoquery` 可查询未安装包，RPM 查询只依赖已安装数据库。看到仓库中存在包不能证明本机已安装；看到二进制文件也要确认它是否受 RPM 管理。

② [知识点] 验证包文件不是自动修复

`rpm -V <PACKAGE>` 将当前文件的大小、校验值、模式、所有者、组等与包元数据比较，无输出通常表示这些可验证属性没有差异。配置文件被合法修改也会报告差异，因此结果是调查线索，不应直接重装覆盖。

③ [边界] 直接 rpm 安装不解析完整仓库依赖

`rpm -ivh file.rpm` 处理本地包但不会像 DNF 一样从仓库自动求解依赖。安装本地 RPM 时优先 `dnf install ./file.rpm`，让 DNF 处理依赖和事务。`--nodeps`、`--force` 会绕过保护，不是常规解法。

[Cheatsheet] 已安装：`rpm -q/qi/ql/qf`；仓库：`dnf info/list/repoquery`；文件完整性线索：`rpm -V`；本地 RPM 优先 `dnf install ./file.rpm`，不绕过依赖。

[操作专题] 配置并证明软件仓库可用

① [知识点] repo 文件最小字段与变量

仓库片段位于 `/etc/yum.repos.d/*.repo`，每个 section ID 必须唯一。`name` 是显示名，`baseurl` 指向包含 repodata 的根，`enabled=1` 启用，`gpgcheck=1` 校验包签名，`gpgkey` 指向可信公钥。

```
[exam-baseos]
name=Exam BaseOS
baseurl=http://content.example.com/rhel9/BaseOS
enabled=1
gpgcheck=1
gpgkey=file:///etc/pki/rpm-gpg/RPM-GPG-KEY-redhat-release
```

`$releasever`、`$basearch` 可由 DNF 展开；在 Shell 中生成文件时必须防止它们被 Shell 提前展开。题目给出固定 URL 时直接使用，不把 RHEL 10 路径代入 RHEL 9。

② [操作点] 写入后用元数据请求验证

<code>dnf repolist --all</code>	# 配置解析与启用状态
<code>dnf clean metadata</code>	# 必要时清理旧元数据
<code>dnf makecache --refresh</code>	# 实际刷新启用仓库
<code>dnf repoquery --repo exam-baseos bash</code>	# 限定仓库查询代表包

`repolist` 显示 ID 不能证明 URL 可达；`makecache --refresh` 才会请求元数据。限定 repo 的查询能证明目标仓库确实提供预期包，但不证明所有包都完整。

③ [验证点] 区分配置、网络和签名层

元数据失败先读取完整错误 URL，检查名称解析、路由、代理、HTTP 状态和 repodata 路径。GPG 错误则检查题目提供的 key、系统时间和包签名，不用 `gpgcheck=0` 作为默认修复。

[Cheatsheet] repo 字段：ID/name/baseurl/enabled/gpgcheck/gpgkey；配置可见 `repolist --all`；真实可用 `makecache --refresh`；来源验证 `repoquery --repo`；签名问题不靠关闭 `gpgcheck`。

[操作专题] 查找、安装、升级和删除软件

① [操作点] 从目标能力反查包

已知描述用 `dnf search`，已知命令或路径用 `dnf provides`。通配模式应引用，避免 Shell 展开。

```
dnf search 'web server'
dnf provides '*/semanage'
dnf provides /usr/bin/rsync
```

查询结果可能包含多个版本或兼容包，应结合 RHEL 9 仓库、架构和目标命令选择，不机械安装第一项。

② [操作点] 事务前读摘要，事务后查数据库

`dnf install <PKG>` 安装，`upgrade <PKG>` 升级指定包，`upgrade` 升级可用包，`remove` 可能连带删除依赖。执行前阅读 Transaction Summary，确认没有意外删除或跨仓库替换。

```
dnf install httpd
rpm -q httpd
rpm -ql httpd | grep -E '/httpd\.service$|/httpd\.conf$'
systemctl cat httpd
```

安装包不会自动保证服务配置、active 或 enabled。包、配置文件和 unit 都确认后，继续按服务章节完成当前与持久状态。

③ [知识点] history 提供事务线索

`dnf history` 列出事务，`dnf history info <ID>` 查看包变化。undo/rollback 可能受当前仓库中旧版本是否仍存在影响，也可能撤销后来依赖变更，不能作为无条件恢复按钮。

④ [验证点] 包、文件、unit 与功能分层

`rpm -q` 证明安装数据库，`rpm -ql/qf` 证明文件归属，`systemctl cat` 证明 unit 入口，应用版本/语法和功能检查证明使用结果。`dnf` 返回成功只覆盖事务层。

[Cheatsheet] 找能力 `search/provides` → 看事务摘要 → `dnf install/upgrade/remove` → `rpm -q/ql/qf` → unit/配置/功能；历史查 `dnf history info`，不盲目 rollback。

[知识专题] 模块流、包组和版本选择边界

① [知识点] AppStream 模块影响可见版本集合

RHEL 9 的部分软件以 AppStream 模块流组织。`dnf module list` 查看流，`module info` 看 profile；启用流会影响依赖求解，安装 profile 会安装一组包。不是所有包都属于模块，且具体可用流由当前 RHEL 9 仓库决定。

② [边界] switch/reset 是有状态操作

改变模块流前先检查已启用流和已安装包。dnf module reset 重置选择，不等于删除已安装内容；切换流可能需要专用 switch-to 流程。考试题未要求版本流时，不额外改变模块状态。

③ [知识点] 包组名称与环境组需查询

dnf group list、group info 查询组，dnf group install '<GROUP>' 安装。显示名可能受语言影响，组 ID 更稳定；不要把一组包安装成功解释为相关服务已配置。

[Cheatsheet] 模块：dnf module list/info；只按题意启用/安装指定流；reset 不删除包；包组先 group list/info，安装后仍按每个服务终态验收。

[操作专题] 配置主机名、默认 target 与 tuned 基线

① [操作点] 主机名同时验证静态值和解析

```
hostnamectl set-hostname servera.example.com
hostnamectl status
getent hosts servera.example.com
```

hostnamectl 证明系统主机名；getent 证明名称解析入口。设置主机名不会自动创建 DNS 或 /etc/hosts 记录。

② [操作点] target 的当前切换与默认值分开

systemctl isolate multi-user.target 当前切换并停止不属于目标依赖的 unit，具有会话中断风险；set-default 只设置下次启动默认 target。分别用 get-default 和当前 unit 状态验证。

③ [操作点] tuned profile 需要推荐值和当前值证据

tuned-adm recommend 给出系统推荐 profile，tuned-adm active 显示当前，tuned-adm profile <NAME> 切换。推荐值不是已经应用；active 也不证明所有工作负载性能达到目标。

④ [验证点] 基础维护不替代服务验收

主机名、target 和 tuned 都是独立系统状态。题目若要求从仓库安装服务，应在这些设置之后仍验证包、unit、监听和功能，避免修改 target 停止刚配置的服务。

[Cheatsheet] 主机名 hostnamectl + getent；默认 target set-default/get-default，当前切换 isolate；tuned recommend/active/profile；基础状态与业务服务分别验收。

[诊断专题] 从仓库错误定位配置、传输、签名或依赖层

① [诊断点] repomd.xml 404 或无法下载

查看错误中的完整 URL，确认 baseurl 是否指向包含 `repodata/repomd.xml` 的根，检查变量展开、DNS、路由和代理。不要反复 clean cache 期待错误 URL 自行修复。

② [诊断点] GPG key 或签名失败

确认 key 来源与题目一致、系统时间正确、包来自期望仓库。关闭 `gpgcheck` 会消除错误但同时移除目标安全属性，不是完成任务。

③ [诊断点] 依赖冲突或包被过滤

用完整 DNF 错误、`repoquery --requires/--whatrequires` 和 `module list` 判断依赖、架构、版本流或仓库缺失。`--allowerase` 可能删除冲突包，执行前必须理解事务摘要，不作为第一尝试。

[Cheatsheet] 404 查 baseurl/repodata；连接错查 DNS/路由/代理；GPG 错查 key/时间/来源；依赖错查 `repoquery/module`；任何放宽选项前先读事务影响。

[经典任务] 从指定仓库安装并验证 Web 服务

配置两个题目给出的 RHEL 9 仓库 `exam-baseos` 与 `exam-appstream`，保持包签名校验。只从这些仓库确认并安装 `httpd`，使服务当前运行且开机启动。主机名设为 `web1.example.com`，但不得假定 DNS 已自动配置。

不得关闭 GPG 校验、使用 `rpm --nodeps` 或无条件允许删除冲突包。验收时证明 repo 文件解析、元数据可达、包来源与安装数据库、关键配置和 unit 文件、服务 active/enabled、监听和本机 HTTP 响应。

请先独立完成。参考解答从下一页开始。

[参考解答] 从仓库元数据推进到服务功能

① [操作点] 写入并刷新仓库

按题目真实 URL 与 key 写 `/etc/yum.repos.d/exam.repo`，保留 `enabled=1`、`gpgcheck=1`。随后：

```
dnf repolist --all
dnf makecache --refresh
dnf repoquery --repo exam-appstream httpd
```

② [操作点] 安装并确认来源对象

```
dnf --disablerepo='*' --enablerepo=exam-baseos,exam-appstream install httpd
rpm -q httpd
rpm -qf /usr/sbin/httpd /usr/lib/systemd/system/httpd.service
rpm -ql httpd | grep -E 'httpd\.conf$|httpd\.service$'
```

限定 repo 只在题目明确要求时使用，并确保依赖仓库同时启用。

③ [操作点] 配置系统状态与服务

```
hostnamectl set-hostname web1.example.com
httpd -t
systemctl enable --now httpd
```

④ [验证点] 完成分层验收

```
hostnamectl status
getent hosts web1.example.com
systemctl is-active httpd
systemctl is-enabled httpd
ss -lntp | grep ':80'
curl -I http://localhost/
```

若 `getent` 没有结果，只能说明名称解析尚未成立，不能撤销已经正确的静态主机名。根据题目另配 DNS 或 hosts。

[Cheatsheet] repo 文件 → `repolist` → `makecache --refresh` → 限定来源 `repoquery` → DNF 事务 → `rpm -q/qf/ql` → 应用语法 → active/enabled/监听/HTTP。

[本章收束] 包事务只是服务终态的中间层

仓库配置决定 DNF 从哪里取得元数据与包，DNF 事务决定安装数据库，RPM 查询把包映射到文件，systemd 和应用工具再验证运行状态。每一层都能局部成功，也都可能留下下一层缺口。

排错时根据错误定位 baseurl、传输、签名、模块或依赖，不用关闭保护或强制删除换取事务通过。最终从仓库、包、文件、unit、监听和功能逐层证明，才能把“已安装”推进为“可用并持久”。